

LAPORAN PRAKTIKUM STRUKTUR DATA



Oleh:

FAWWAZ KHALID 2511532004

MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU : DR. WAHYUDI, S.T, M.T

ASISTEN LABOR : M GALID

FAKULTAS TEKNOLOGI INFORMASI

PEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, 2026

KATA PENGANTAR

Pedoman ini disusun sebagai rujukan resmi bagi mahasiswa Departemen Informatika dalam penyusunan laporan praktikum pada mata kuliah Pemrograman Dasar dengan Java. Dokumen ini tidak hanya memberikan gambaran umum mengenai format penulisan, tetapi juga menguraikan secara rinci sistematika laporan, tata cara penyajian isi, serta contoh penulisan kode program yang dilengkapi dengan referensi ilmiah. Melalui panduan ini, mahasiswa diharapkan mampu menyusun laporan yang tidak sekadar memenuhi aspek administratif, tetapi juga mencerminkan ketelitian, keteraturan, dan penerapan kaidah penulisan akademik pada tingkat dasar. Dengan demikian, laporan praktikum yang dihasilkan dapat berfungsi sebagai media pembelajaran, dokumentasi kegiatan, sekaligus sarana untuk melatih keterampilan menulis ilmiah yang akan bermanfaat dalam jenjang studi selanjutnya.

Padang, 2025

Tim Penyusun

FAWWAZ KHALID

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Pengertian Praktikum.....	1
1.2 Latar Belakang Praktikum.....	1
1.3 Rumusan Masalah.....	1
1.4 Tujuan Praktikum.....	1
BAB II PENULISAN LAPORAN PRAKTIKUM.....	2
2.1 Alat dan Bahan.....	2
2.2 Landasan Teori.....	2
2.3 Langkah Kerja.....	2
2.4 Hasil dan Analisis.....	3
BAB III KESIMPULAN.....	4
3.1 Ringkasan.....	4
DAFTAR PUSTAKA.....	4
BAB I	

PENDAHULUAN

1.1 Pengertian ADT

Tipe Data Abstrak (Tda) Atau Lebih Dikenal Dalam Bahasa Inggris Sebagai **Abstract Data Type (Adt)** Merupakan Model Matematika Yang Merujuk Pada Sejumlah Bentuk Struktur Data Yang Memiliki Kegunaan Atau Perilaku Yang Serupa; Atau Suatu Tipe Data Dari Suatu Bahasa Pemrograman Yang Memiliki Semantik Yang Serupa. Tipe Data Abstrak Umumnya Didefinisikan Tidak Secara Langsung, Melainkan Hanya Melalui Operasi Matematis Tertentu Sehingga Membutuhkan Penggunaan Tipe Data Tersebut Meski Dengan Risiko Kompleksitas Yang Lebih Tinggi Atas Operasi Tersebut.

BAB II PENULISAN LAPORAN PRAKTIKUM

2.1 Alat dan Bahan

1. Perangkat baik komputer maupun laptop
2. Teks editor atau *IDE* (misalnya NetBeans, Eclipse, atau IntelliJ IDEA)

2.2 Landasan Teori

1. Tipe data sebuah variabel adalah kumpulan nilai yang dapat dimuat oleh variabel ini. Misalnya sebuah tipe boolean hanya bernilai TRUE atau FALSE, tidak boleh nilai yang lain. TDA adalah suatu model matematika, disertai sekumpulan operasi terhadap model itu

2.3 Langkah Kerja

1. Membuat projek JAVA
2. Membuat file baru di dalam Projek JAVA tersebut
3. Masukkan kode yang ingin dijalankan contoh kode yang dikerjakan di pekan 5:

```

1 package pekan8_2511532004;
2
3 public class mergesort_2511532004 {
4
5
6     void merge_2004(int arr_2004[], int l_2004, int m_2004, int r_2004) {
7
8         int n1_2004 = m_2004 - l_2004 + 1;
9         int n2_2004 = r_2004 - m_2004;
10
11         /* Create temp arrays */
12         int L_2004[] = new int[n1_2004];
13         int R_2004[] = new int[n2_2004];
14
15         /* Copy data to temp arrays */
16         for (int i_2004 = 0; i_2004 < n1_2004; ++i_2004)
17             L_2004[i_2004] = arr_2004[l_2004 + i_2004];
18
19         for (int j_2004 = 0; j_2004 < n2_2004; ++j_2004)
20             R_2004[j_2004] = arr_2004[m_2004 + 1 + j_2004];
21
22         int i_2004 = 0, j_2004 = 0;
23
24         /* Initial index of merged subarray array */
25         int k_2004 = l_2004;
26
27         while (i_2004 < n1_2004 && j_2004 < n2_2004) {
28             if (L_2004[i_2004] <= R_2004[j_2004]) {
29                 arr_2004[k_2004] = L_2004[i_2004];
30                 i_2004++;
31             } else {
32                 arr_2004[k_2004] = R_2004[j_2004];
33                 j_2004++;
34             }
35             k_2004++;
36         }
37
38         /* Copy remaining elements of L[] if any */
39         while (i_2004 < n1_2004) {
40             arr_2004[k_2004] = L_2004[i_2004];
41             i_2004++;
42             k_2004++;
43         }

```

```

        arr_2004[k_2004] = R_2004[j_2004];
        j_2004++;
        k_2004++;
    }
}

void sort(int arr_2004[], int l_2004, int r_2004) {
    if (l_2004 < r_2004) {
        // Find the middle point
        int m_2004 = (l_2004 + r_2004) / 2;

        // Sort first and second halves
        sort(arr_2004, l_2004, m_2004);
        sort(arr_2004, m_2004 + 1, r_2004);

        // Merge the sorted halves
        merge_2004(arr_2004, l_2004, m_2004, r_2004);
    }
}

// A utility function to print array of size n
static void printArray(int arr_2004[]) {
    int n_2004 = arr_2004.length;

    for (int i_2004 = 0; i_2004 < n_2004; ++i_2004)
        System.out.print(arr_2004[i_2004] + " ");

    System.out.println();
}

public static void main(String args_2004[]) {

    int arr_2004[] = {12, 11, 13, 5, 6, 7};

    System.out.println("Sebelum terurut");
    printArray(arr_2004);

    mergesort_2511532004 ob_2004 = new mergesort_2511532004();
    ob_2004.sort(arr_2004, 0, arr_2004.length - 1);

    System.out.println("\nSesudah Terurut menggunakan Merge Sort");
    printArray(arr_2004);
}
}

```

```
<terminated> mergesort_2511532004 [Java Application] C:\Users\lenovo\p2\pool\plugins\
Sebelum terurut
12 11 13 5 6 7

Sesudah Terurut menggunakan Merge Sort
5 6 7 11 12 13
```

```
package pekan8_2511532004;

public class quicksort_2511532004 {

    static void swap_2004(int[] arr_2004, int i_2004, int j_2004)
    {
        int temp_2004 = arr_2004[i_2004];
        arr_2004[i_2004] = arr_2004[j_2004];
        arr_2004[j_2004] = temp_2004;
    }

    // Metode tambahan untuk mengatur pivot menggunakan Median-of-Three
    static void medianOfThree_2004(int[] arr_2004, int low_2004, int high_2004)
    {
        int mid_2004 = low_2004 + (high_2004 - low_2004) / 2;

        // Urutkan elemen low, mid, dan high
        if (arr_2004[low_2004] > arr_2004[mid_2004]) {
            swap_2004(arr_2004, low_2004, mid_2004);
        }

        if (arr_2004[low_2004] > arr_2004[high_2004]) {
            swap_2004(arr_2004, low_2004, high_2004);
        }

        if (arr_2004[mid_2004] > arr_2004[high_2004]) {
            swap_2004(arr_2004, mid_2004, high_2004);
        }

        swap_2004(arr_2004, mid_2004, high_2004);
    }

    static int partition_2004(int[] arr_2004, int low_2004, int high_2004)
    {
        // Panggil fungsi medianOfThree sebelum menentukan pivot
        medianOfThree_2004(arr_2004, low_2004, high_2004);

        int pivot_2004 = arr_2004[high_2004]; // Sekarang arr[high] sudah berisi nilai median
        int i_2004 = (low_2004 - 1);

        for (int j_2004 = low_2004; j_2004 <= high_2004 - 1; j_2004++) {
            // Jika elemen saat ini lebih kecil dari atau sama dengan pivot
            if (arr_2004[j_2004] < pivot_2004) {
                return (i_2004 + 1);
            }
        }

        static void quicksort_2511532004(int[] arr_2004, int low_2004, int high_2004)
        {
            if (low_2004 < high_2004) {
                int pi_2004 = partition_2004(arr_2004, low_2004, high_2004);
                quicksort_2511532004(arr_2004, low_2004, pi_2004 - 1);
                quicksort_2511532004(arr_2004, pi_2004 + 1, high_2004);
            }
        }

        public static void printArr_2004(int[] arr_2004)
        {
            for (int i_2004 = 0; i_2004 < arr_2004.length; i_2004++) {
                System.out.print(arr_2004[i_2004] + " ");
            }
            System.out.println();
        }

        public static void main(String[] args_2004)
        {
            int[] arr_2004 = { 10, 7, 8, 9, 1, 5 };
            int N_2004 = arr_2004.length;

            System.out.println("Data sebelum diurutkan: ");
            printArr_2004(arr_2004);

            quicksort_2511532004(arr_2004, 0, N_2004 - 1);

            System.out.print("Data Terurut quicksort: ");
            printArr_2004(arr_2004);
        }
    }
}
```

```
Data sebelum diurutkan:
10 7 8 9 1 5
Data Terurut quicksort: 1 5 7 8 9 10
```

```

1 package pekan8_2511532004;
2
3 public class shellsort_2511532004 {
4
5
6     public static void shellSort(int[] A_2004) {
7         int n_2004 = A_2004.length;
8         int gap_2004 = n_2004 / 2;
9
10        while (gap_2004 > 0) {
11            for (int i_2004 = gap_2004; i_2004 < n_2004; i_2004++) {
12                int temp_2004 = A_2004[i_2004];
13                int j_2004 = i_2004;
14
15                while (j_2004 >= gap_2004 && A_2004[j_2004 - gap_2004] > temp_2004) {
16                    A_2004[j_2004] = A_2004[j_2004 - gap_2004];
17                    j_2004 = j_2004 - gap_2004;
18                }
19
20                A_2004[j_2004] = temp_2004;
21            }
22            gap_2004 = gap_2004 / 2;
23        }
24    }
25
26    public static void main(String[] args_2004) {
27        int[] data_2004 = {3, 10, 4, 6, 0, 9, 7, 2, 1, 5};
28
29        System.out.print("Sebelum : ");
30        printArray(data_2004);
31
32        shellSort(data_2004);
33    }
34 }

```

<terminated> shellsort_2511532004 [Java Application] C:\Users\tenovo\p2\poo\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v20250724-1412\jre\bin\ja
 Sebelum : 3 10 4 6 0 9 7 2 1 5
 Sesudah (Shell Sort) : 0 1 2 3 4 5 6 7 9 10

```

1 import java.awt.BorderLayout;
2 import java.awt.Color;
3 import java.awt.Dimension;
4 import java.awt.EventQueue;
5 import java.awt.FlowLayout;
6 import java.awt.Font;
7
8 import javax.swing.BorderFactory;
9 import javax.swing.JButton;
10 import javax.swing.JFrame;
11 import javax.swing.JLabel;
12 import javax.swing.JPanel;
13 import javax.swing.JPasswordField;
14 import javax.swing.JTextField;
15
16 import pekan7_2511532004;
17
18 public class bubbl_2511532004 {
19     private static JFrame frame;
20     private JPanel panel;
21     private int[] data;
22     private JLabel label;
23     private JButton button;
24     private JTextField textField;
25     private JPasswordField passwordField;
26
27     private int i_2004 = 1, j_2004;
28     private boolean sorting_2004 = false;
29     private int stepCount_2004 = 1;

```

Insertion Sort Langkah per Langkah

Masukkan angka (pisahkan dengan koma): 3,5,6

Langkah 1: Memasukkan 5
 Hasil: 3, 5, 6

Langkah 2: Memasukkan 6
 Hasil: 3, 5, 6

Langkah Selanjutnya

Reset

2.4 Hasil dan Analisis

Jika kode berjalan seperti yang kita inginkan maka:

1. Menyimpan file
2. Mencatat hasil output

Jika kode gagal maka:

1. Mencarik kode yang salah lalu coba jalankan ulang

BAB III KESIMPULAN

3.1 Ringkasan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. Deklarasi variabel pada Java harus disesuaikan dengan tipe data
2. Bisa menggunakan queue , stack, linkedlist dan arraylist untuk fungsi berbeda

DAFTAR PUSTAKA

1. Tipe data abstrak-wikipedia
https://id.wikipedia.org/wiki/Tipe_data_abstrak 4